



Stos technologiczny *i status projektu*

Aplikacja webowa wspomagająca planowanie aktywności fizycznej z uwzględnieniem jakości powietrza

Piotr Pawlus · Bartosz Ziółkowski · Szymon Ziędalski

Politechnika Warszawska · kwiecień 2026

Co przedstawimy

01

Czym jest BikAir?

Krótkie przypomnienie celu i działania systemu

02

Architektura systemu

Frontend, backend, źródła danych — jak to się komunikuje

03

Stos technologiczny

Jakich narzędzi używamy i dlaczego właśnie ich

04

Status prac

Co już mamy, nad czym pracujemy, co jeszcze przed nami

05

Wyzwania i kolejne kroki

Ryzyka techniczne i plan na najbliższe tygodnie

Czym jest BikAir?

Aplikacja webowa, która wyznacza trasy rowerowe i biegowe optymalizowane pod kątem jakości powietrza.

Użytkownik wskazuje punkt startu i celu. Algorytm — zamiast wyznaczać tylko najkrótszą trasę — uwzględnia również poziom zanieczyszczeń (PM2.5, PM10) i proponuje wariant „najzdrowszy”.



Real-time AQI



Wyznaczanie tras



Zdrowie i ekologia

Jak to działa

1

Użytkownik podaje punkt A i B

2

Backend pobiera dane AQI z Airly i pobiera graf dróg z OSM

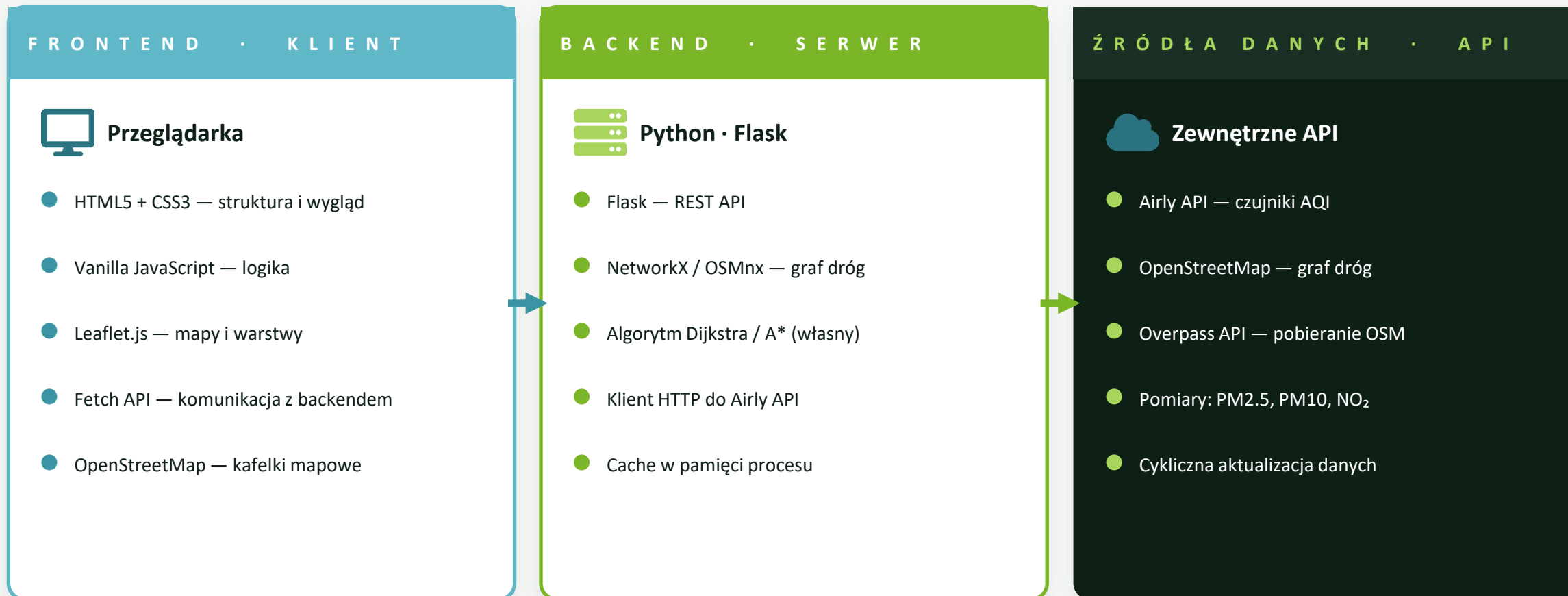
3

Algorytm Dijkstra/A* z wagami uwzględniającymi zanieczyszczenia

4

Frontend pokazuje trasę + warstwę AQI na mapie Leaflet

Architektura systemu



Frontend — co i dlaczego



HTML5 + CSS3

Standard, bez frameworka

Świadomy wybór: zero zależności, zero konfiguracji, pełna kontrola nad DOM. W naszej skali (kilka ekranów) framework byłby nadmiarowy.



Vanilla JavaScript

Lekkość i prostota

Brak React/Vue/Angular oznacza mały bundle, brak build-stepu i łatwiejszy debug. Logika frontendu sprowadza się głównie do wywołań API i renderu mapy.



Leaflet.js

Standard map open-source

Lekka biblioteka (~40 KB), świetna dokumentacja, natywne wsparcie dla GeoJSON. Idealna do warstw tras + buforów AQI bez kosztów Mapbox/Google.



OpenStreetMap

Darmowe kafelki mapowe

Brak limitów zapytań, dane społecznościowe, pełna licencja na użycie komercyjne. Spójność: te same dane OSM zasilają nasz routing po stronie backendu.

Backend — co i dlaczego



Python 3

Język danych przestrzennych

Najbogatszy ekosystem do GIS i grafów (NetworkX, OSMnx, Shapely). Czytelny kod, szybkie prototypowanie. Naturalny wybór dla aplikacji łączącej dane z różnych API.



Flask

Mikroframework REST

Minimalne API, brak zbędnych warstw. Definiowanie endpointów w kilku liniach, wbudowany serwer dev, łatwa integracja z bibliotekami GIS. Mniejszy narzut niż Django.



NetworkX + OSMnx

Graf dróg z OSM

OSMnx pobiera siatkę dróg jako graf NetworkX. Mamy gotowe narzędzia do Dijkstry/A*, możemy modyfikować wagi krawędzi w oparciu o lokalne AQI.



Airly API

Polskie dane jakości powietrza

Gęsta sieć czujników, szczególnie w polskich miastach. Darmowy plan dla projektów edukacyjnych, REST + JSON, dane PM2.5 / PM10 / NO₂ z aktualizacją co kilka minut.

Narzędzia wspierające



Git + GitHub

Wersjonowanie i współpraca zespołowa



VS Code

Edytor — Python, JS, debug, Live Server



Figma (makiety)

Inspiracje UI, choć makiety robimy w HTML



Postman

Testowanie endpointów Airly i własnego API

DLACZEGO TAKI STOS?

01

Open-source od końca do końca

Brak kosztów licencji, brak vendor lock-in

02

Małe zależności = mały dług techniczny

Każda biblioteka ma jasne uzasadnienie

03

Edukacyjna wartość projektu

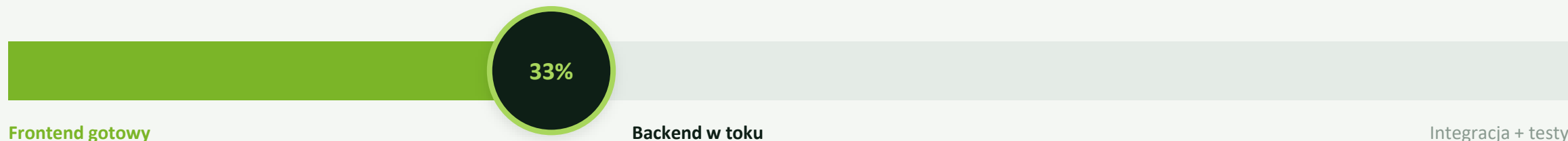
Pisanie własnego routingu uczy więcej niż użycie OSRM

04

Łatwa demonstracja

Frontend działa w przeglądarce, backend lokalnie w Pythonie

Status prac — gdzie jesteśmy



ZROBIONE

- Makiety UI (HTML/CSS) — 4 ekrany × mobile + desktop
- Frontend statyczny: ekran startowy, mapa, statystyki, nawigacja
- System kolorów, typografia, identyfikacja wizualna
- Specyfikacja wymagań F1–F3 i нефunkcjonalnych
- Konfiguracja repozytorium GitHub i podziału prac

W TRAKCIE

- Setup Flaska — struktura projektu, endpointy
- Pobieranie grafu dróg z OSM przez OSMnx
- Klient Airly API — autoryzacja i mapowanie pomiarów
- Prototyp Dijkstry z wagami zależnymi od AQI
- Logika łączenia czujników z węzłami grafu

? PRZED NAMI

- Pełna integracja frontend ↔ backend (REST API)
- Renderowanie tras alternatywnych z porównaniem
- Wizualizacja bufforów AQI na mapie
- Testy wydajnościowe (cel: trasa < 3 s)
- Dokumentacja końcowa i deployment

Wyzwania i kolejne kroki



Wyzwania techniczne

● Mapowanie czujników na graf

Czujniki Airly nie leżą dokładnie na drogach OSM — trzeba interpolować ich wpływ na okoliczne krawędzie grafu (buffory + wagi).

● Wydajność algorytmu

Graf miasta ma dziesiątki tysięcy węzłów. Dijkstra na pełnym OSM może być wolna — rozważamy A* z heurystyką lub redukcję grafu.

● Niska gęstość czujników

W mniejszych miejscowościach Airly ma kilka stacji na cały obszar. Dlatego zdecydowaliśmy się na konkretny teren (Warszawa i okolice Krakowa)

● Limity Airly API

Plan darmowy ma limity zapytań — potrzebujemy cachowania i rozsądnej polityki odświeżania (co 5–20 min, nie częściej).



Plan na najbliższe tygodnie

Tydz. 1–2

Działający endpoint /route — przyjmuje A/B, zwraca trasę z OSM

Tydz. 3

Integracja Airly + przeliczanie wag krawędzi grafu

Tydz. 4

Frontend ↔ backend: rendering trasy + warstwy AQI w Leaflet

Tydz. 5

Trasy alternatywne, statystyki, ekran nawigacji

Tydz. 6

Testy, optymalizacja, dokumentacja, prezentacja prototypu

Dziękujemy

za uwagę



Piotr Pawlus · Bartosz Ziółkowski · Szymon Ziędalski